

Queues

instructor → main speaker teacher

↳ lady → Sachin Sachdeva



Notes

- Introduction to Queues
- Implementation of Queues
 - Using Arrays
 - Using Linked List
 - Using Stacks
- Ath Perfect Number
- Doubly Ended Queue
- Sliding Window Maximum





Queue

Linear data structure where elements are inserted on one end & removed from other end.

FIFO (First In First Out)



Some Real World examples :

- Customer Support Call
- Order at Restaurant
- Toll Plaza
- Message Queues
- Printer



● → java

Operations in Queues

1. Enqueue(x) → Insert element at REAR end.
`q.add(x)`
2. Dequeue() → Remove element from FRONT end & return it.
`q.remove()`
3. front() / peek() → Tells the element which is present at FRONT of the queue.
`q.peek()`
4. size() → Returns no of elements in queue.
`q.size()`

T.C : $O(1)$

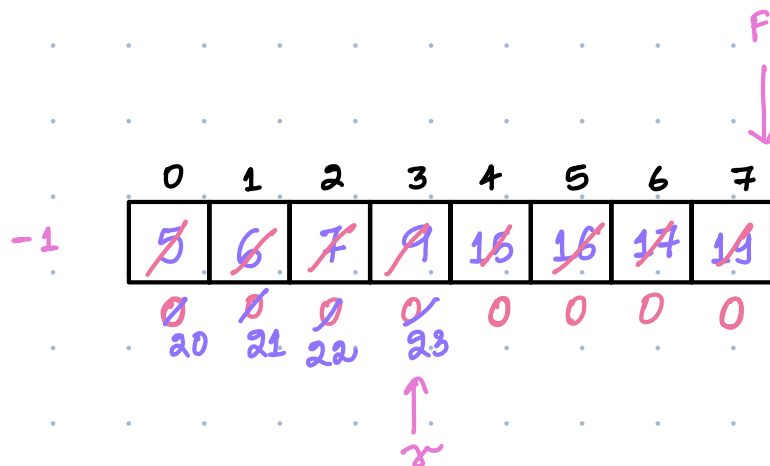


Array Implementation of Queues → 2 pointers

$f \rightarrow \text{front}$
 $r \rightarrow \text{rear}$

$f = -1 \rightarrow$ Idx of the last element which is just Removed

$r = -1 \rightarrow$ " " " " " " " " Inserted.



Add (5)

Add (6)

Add (7)

Add (9)

remove()

remove()

remove()

remove()

remove()

Add (15)

Add (16)

Add (17)

Add (19)

Add (20)

Add (21)

Add (22)

Add (23)

Add (24)

remove()

remove()

remove()

remove()

remove()

$(f == r)$

- Queue is Empty
- Queue is Full

{ Size + modulo is the key } to traverse



```
void add (int x) {
```

```
    if (size == N) return;
```

```
    r = (r + 1) % N;
```

```
    A[r] = x;
```

```
    size++;
```

```
    if (r == -1) { r = 0; }
```

```
}
```

This checks makes sure that we add element only if space is there

as r represents last element which is added r+1 gives next element. This check + arr size checks says if size is there & r has gone overflow that means at 0th idx space will be free insert there

```
int remove() {
```

```
    if (f == r) return -1;
```

```
    if (size == 0) return -1;
```

```
    int val = A[f];
```

```
    f = (f + 1) % N;
```

```
    A[f] = 0;
```

```
    size--;
```

```
    return val;
```

```
}
```

```
int peek() {
```

```
    if (size == 0) return -1;
```

```
    val = (f + 1) % N;
```

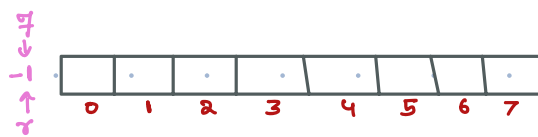
```
    return A[val];
```

```
}
```

```
int size() {
```

```
    return size;
```

```
}
```



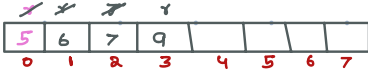
Add (5)



Add (6)

Add (7)

Add (9)



remove ()

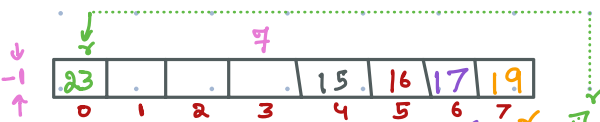
remove ()

remove ()

remove ()

remove () → error

$f = f + 1$
 $val = arr[f]$



Add (15)

Add (16)

Add (17)

Add (19)

Add (23) → where will you add ??

↳ space is there at 0th index

↳ $r = (r + 1) \% N$ & $size \neq 0$



FIFO

Implementation of Queue using LinkedList

- Insert At Tail, Remove From Head ✓
 $O(1)$ $O(1)$
 - Insert At Tail, Remove From Tail
 - Insert At Head, Remove From Head
 - Insert At Head, Remove From Tail ✗
 $O(1)$ $O(N)$ in SL
 $O(1)$ in DL
- Doesn't follow FIFO

QUIZ

What will be the state of the queue after these operations

~~enqueue(3)~~, ~~enqueue(7)~~, ~~enqueue(12)~~, ~~enqueue()~~, ~~enqueue()~~, ~~enqueue(8)~~, ~~enqueue(3)~~

~~3~~ ~~7~~ 12 8 3

QUIZ

What will be the state of the queue after these operations

~~enqueue(4)~~, ~~enqueue()~~, ~~enqueue(9)~~, ~~enqueue(3)~~, ~~enqueue(7)~~, ~~enqueue(11)~~, ~~enqueue(20)~~, ~~enqueue()~~

~~4~~ ~~9~~ 3 7 11 20



Implementation of Queue using Stack - [Directi]

→ Dequeue

→ Enqueue

→ front()

behind the scene only these functions can be used

→ push
→ pop
→ top / peek
→ isEmpty
→ size

~~5~~ ~~6~~ ~~7~~ ~~9~~ 17 19

Add (5)

Add (6)

Add (7)

Add (9)

Remove() → 5

Remove() → 6

Add (17)

Add (19)

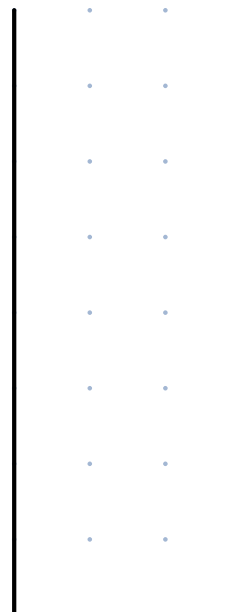
Remove() → 7

Remove() → 9

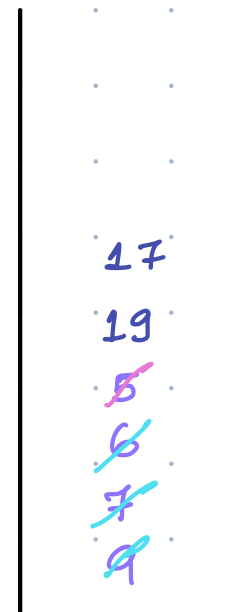
Remove()

is it present at the bottom of the stack then how

↓
have one more stack



St1



St2

1st Remove: Move 4 ele from St1 → St2 & St2.pop()
4 push + 4 pop + 1 pop = 9 operⁿ

2nd Remove: St2.pop() = 1 operⁿ

3rd Remove: St2.pop() = 1 operⁿ

4th Remove: St2.pop() = 1 operⁿ



4 Remove : 12 operⁿ

1 Remove : $\frac{12}{4} = 3$ operⁿ

T.C of Remove : $O(1)$ Amortised

add(x)

→ st1.push(x)

→ $O(1)$

remove()

if (st2.size() == 0)

Yes

No

return st2.pop();

while (st1.size() != 0) {

st2.push(st1.pop());

}

return st2.pop();

Amortised
 $O(1)$

peek()

if (st2.size() == 0)

Yes

No

return st2.top();

while (st1.size() != 0) {

st2.push(st1.pop());

}

return st2.top();

Amortised
 $O(1)$

size()

return (st1.size() + st2.size()) → $O(1)$

Dry run of above

Add (5) → insert in stack 1

Add (6) → insert in stack 1

Add (7) → insert in stack 1

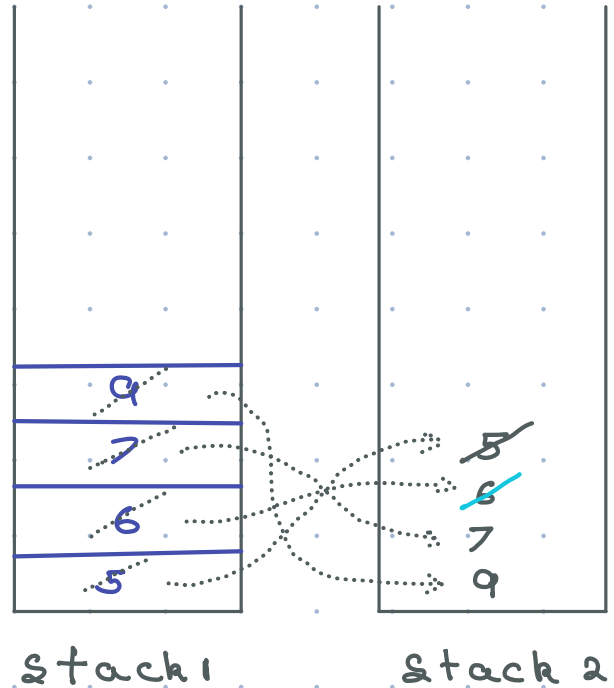
Add (9) → insert in stack 1

Remove() → 5 → now you want to remove the 5 which is at the bottom of

Remove() → 6

stack 1 so what to do

↳ shift all element from stack 1 to stack 2 → now 5 is at top in stack 2 POP in stack 2



Add (17) → add to S_1

Add (19) →

Remove() → 7 → check S_2 is empty

Remove() → 9

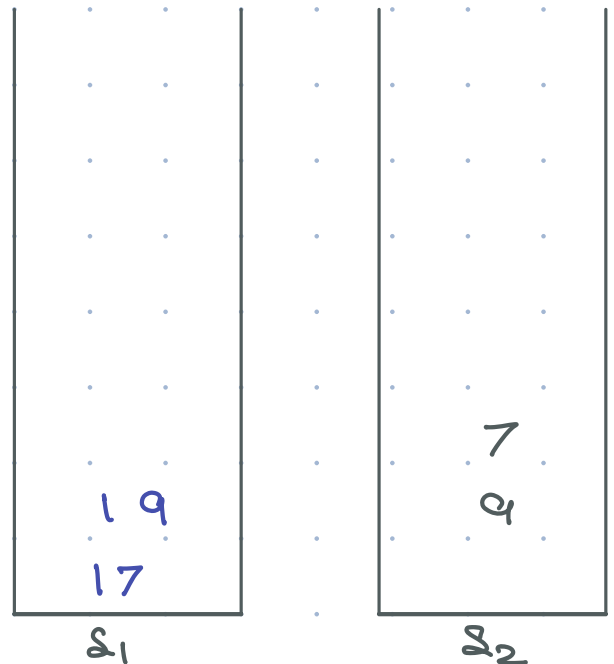
Remove()

yes

Shift all the element from S_1 to S_2

no

$S_2 \cdot \text{pop}()$



↳ check S_2 is empty → no → $S_2 \cdot \text{pop}()$

yes → no removal



Perfect Numbers

< Question > : Find Ath perfect number.

A perfect number has some properties →

1. Digits of the number can be 1 or 2 or both.
2. Number of digits in perfect number must be **even**.
3. Number must be **palindrome**.

$$1 \leq N \leq 10^5$$

$$A = 7 \rightarrow 1111111$$

11

22

1111

1221

2112

2222

1111111

Observation:

$$n = a a'$$

= first half + reverse (first half).

If we know 1st half, we can generate complete no.

1st Halfs :-

1 digit:

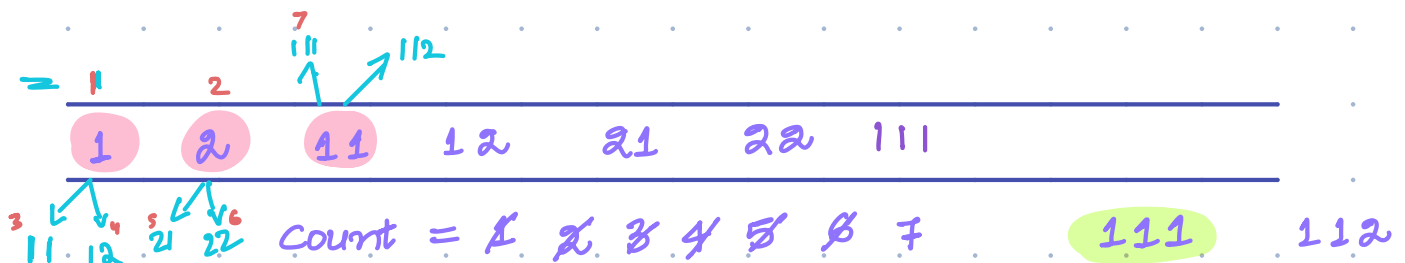
2 digit:

3 digit:



count =

A = 7





$$= 111111$$

</> Code

```

count = 0;    ans = " ";
if (A == 1)    return "11";
if (A == 2)    return "22";

```

```

Queue < String > q;
q.insert("1");
count++;
q.insert("2");
count++;

```

```

while (true) {

```

```

    // Remove element from queue

```

```

    String num = q.remove();

```

```

    // Compute a new numbers from num.

```

```

    String first-no = num + "1";

```

```

    count++;

```

```

    if (count == A) {

```

```

        | ans = first-no;
        | break;
    }

```

```

    q.add(first-no);

```

```

    String second-no = num + "2";

```

```

    count++;

```

```

    if (count == A) {

```

```

        | ans = second-no;
        | break;
    }

```

$$A = 7$$

~~1~~ ~~2~~ ~~1~~ 12 21 22

count = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ 7

ans = " " 111

num = ~~1~~ ~~2~~ 11

first-no = ~~1~~ ~~2~~ 111

second-no = ~~1~~ 22

(A-1) insertions
A/2 removal

T.C: O(A)

$$A - 1 + A/2$$

$$= 3A/2 - 1$$

S.C: O(A)

q. add (second-no) %

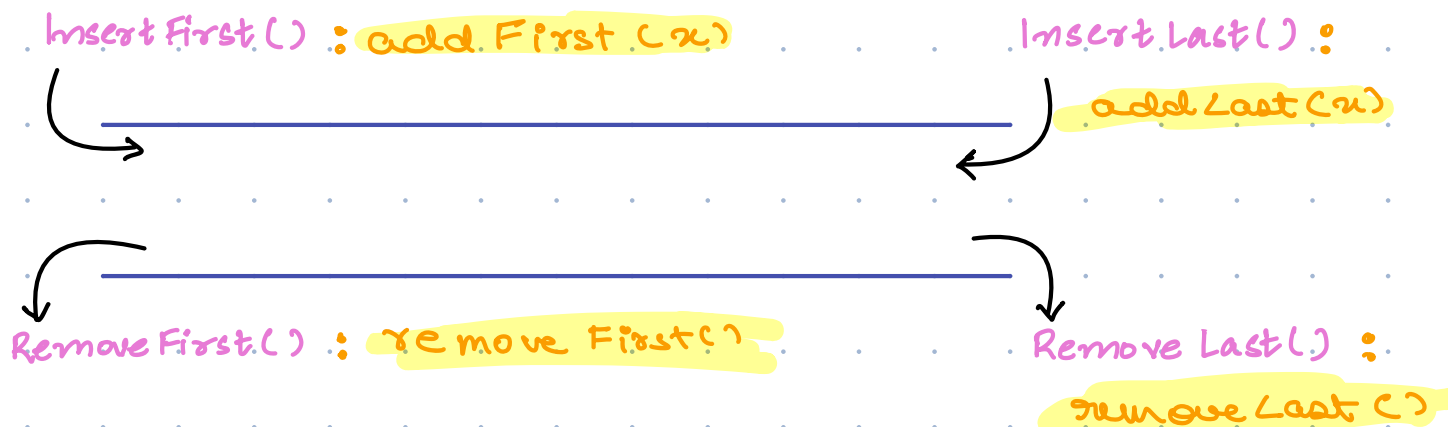
3

return (ans + reverse(ans)) %



Doubly Ended Queue (Deque)

- Data structure where we can access elements from both ends.
- Add/Remove from both sides.



- All these are $O(1)$ operation.
- Deque can act as both STACK / QUEUE.

Insert / Remove
from same end
(either HEAD or
TAIL)

Insert / Remove
from different
ends.



Sliding Window Maximum

< Question > : Find max of every subarray of size K.

[3 , 2 , 3 , 4 , 5 , 5 , 4 , 5 , 6]
0 1 2 3 4 5 6 7 8

K = 4

[0 - 3] → 4

$1 \leq N \leq 10^5$

[1 - 4] → 5

[2 - 5] → 5

[3 - 6] → 5

[4 - 7] → 5

[5 - 8] → 6

QUIZ

Given an integer array A and an window of size k find the max element. A = [1, 4, 3, 2, 5]

k = 3

[4, 4, 5]



BF Idea

 $(N - k + 1)$ subarrays.

Generate all subarrays of size $= k$. Iterate on each subarray & get max.



Why simple Sliding Window doesn't work?

0	1	2	3	4	5	6	7	8	9
3	15	6	12	4	2	10	9	13	2

 $k = 4$

Max = 15

Max = 15

15 is no longer MAX as it is not part of window &

to compute MAX, we need to iterate on this complete window.

Worst case occurs when this situation occurs for every window as is the case when elements are present in DECREASING ORDER.

Eg: $\{5, 4, 3, 2, 1\}$

Total no of subarray $= N - k + 1$

↳ no of iterations $= (N - k + 1) \times k$

T.C: $O(N^2)$

at $k = \frac{N}{2}$ T.C $\Rightarrow N^2$



Optimised Approach: Sliding window + contribution technique

This is similar to Nearest Smaller element on left que with slight twist.

0	1	2	3	4	5	6	7	8	9
3	15	6	12	4	2	10	9	13	2

$K=4$

Max = 15

Max = 15

Max = 12

Max = 12

Max = 10

Max = 13

Max = 13

3	15	6	12	4	2	10	9	13	2
--------------	---------------	--------------	---------------	--------------	--------------	----	---	----	---

MAGIC BOX $\rightarrow$ Deque.

T.C: $O(N)$

S.C: $O(1)$

2 parts $\rightarrow$ building the window

$\rightarrow$ sliding the window



there are 2 cases for a window

↳ in first window $\Rightarrow$ we compute max which is 15

0	1	2	3
3	15	6	12

Max 15

↳ but when we move window to next

0	1	2	3
3	15	6	12

Max

↳ we have to check

- element getting removed is not the max
- if yes then what was the largest compare that with new element

building the window part

0	1	2	3	4	5	6	7	8	9
3	15	6	12	4	2	10	9	13	2

windows

$K=4$

3, 15, 6, 12

15, 6, 12, 4

6, 12, 4, 2

12, 4, 2, 10

4, 2, 10, 9

$i \rightarrow i \rightarrow i \rightarrow i$

3	15	6	12
--------------	----	--------------	----

as 6 is coming after 15 even though it is smaller it can be the larger for next when 15 gets removed while sliding - eg $\rightarrow$ 6, 12, 4, 2 $\rightarrow$ 15 is gone

① at $i=0$

box is empty

so add it

② as $15 > 6$ all the next windows while sliding will use 15 as the max but no one will use 6 as max

④ at 12 is > 6 & before also of 6

remove 6

$\Rightarrow$ so first window :

15	12
----	----

↑ see largest

↓ largest



Approach:

1 For 1st window, we follow same process as in Nearest Greater Element.

1.1 • Pop all smaller elements. `RemoveFromRear()`

1.2 • Insert current element. `InsertAtRear()`

Elements in MAGIC BOX will be in DESCENDING ORDER.

1.3 At the end, element at front will be ANS for 1st window.

`ans[0] = Peek()`

2 For Remaining Windows,

2.1 • Check if element at start of MAGIC BOX still part of current window. If not remove it. `RemoveFromFront()`

2.2 • Pop all smaller elements. from rear

2.3 • Insert current element. → in rear

2.4 • Update ans. (element at front will be ANS)

NOTE: Since we need to do operⁿ on both ends so the datastructure to be used is DEQUE.



</> Code

```
void maxwindow (int[] A, int k) {
```

```
    Deque<Integer> dq;
```

```
    // Insert 1st window
```

```
    ① for (i=0; i<k; i++) {
```

```
        ①.1 while (dq.size() > 0 && A[i] > dq.getLast()) {
```

```
            |   dq.removeLast();
            |   }
        }
```

```
        ①.2 dq.addFirst(A[i]); } dq.addLast(A[i]);
```

This is wrong

it should be `dq.addLast(A[i]);`

```
        1.3 print (dq.getFirst());
```

```
    // For all other window
```

```
    s=1, e=k;
```

```
    ② while (e < n) {
```

```
        // Remove A[s-1] if not part of current window.
```

```
        2.1 if (A[s-1] == dq.getFirst())
            dq.removeFirst();
```

```
        // Pop smaller elements
```

```
        2.2 while (dq.size() > 0 && A[e] > dq.getLast()) {
```

```
            |   dq.removeLast();
            |   }
        }
```

```
        2.3 dq.addFirst(A[e]); dq.addLast(A[e]);
```

This should be add last

```
        2.4 print (dq.getFirst());
```

```
        s++, e++;
```

```
    }
```

```
}
```


Scenario:

At TechTrade Inc., a trading team focuses on day-trading technology stocks. They employ a strategy that involves selling stocks at short-term peaks to maximize profits. The team uses an algorithm to determine the best time to sell stocks based on minute-to-minute price data.

Stock Prices Array A

Consider the minute-by-minute stock prices of a tech company, TechCorp, over a 10-minute interval:

$A = [220, 215, 230, 225, 240, 235, 230, 245, 250, 240]$